

Introducción a Arquitectura de las Computadoras

Diego Feroldi

Arquitectura del Computador*
Departamento de Ciencias de la Computación
FCEIA-UNR



* Actualizado 12 de agosto de 2026 (D. Feroldi, feroldi@fceia.unr.edu.ar)

Índice

1. Definición de computadora	1
2. Concepto de Arquitectura del Computador	1
3. Arquitectura de von Neumann	2
4. Modelo de ejecución	4
5. Organización básica de una computadora	6
5.1. Unidad Central de Procesamiento (CPU)	6
5.2. Registros	8
5.3. Contador de programa	9
5.4. Unidad Aritmética Lógica (ALU)	9
5.5. Buses	11
5.6. Memoria principal	12
5.7. Memoria Caché	12
5.8. Memoria secundaria	13
5.9. Dispositivos de entrada/salida	14
6. Jerarquía de memoria	15
7. Clasificación de arquitecturas	15
8. Microarquitectura	17
9. Breve historia de las computadoras	18
9.1. Computadoras mecánicas	18
9.2. Computadoras con válvulas de vacío	19
9.3. Computadoras con transistores	21
9.4. Computadoras con circuitos integrados	23
9.5. Computadoras con integración a muy gran escala	23
A. Apéndices	26
A.1. Primeros lenguajes de programación	26

1. Definición de computadora

“Una computadora digital es una máquina que puede resolver problemas ejecutando las instrucciones que recibe de las personas”.

Andrew S. Tanenbaum

Organización de computadoras: un enfoque estructurado, 2000.

La definición anterior es solo una entre muchas posibles de lo que es una computadora. Esta en particular sugiere que, a pesar de las enormes prestaciones de las computadoras actuales, en su nivel más elemental estas máquinas realizan operaciones muy simples. De hecho, es importante destacar que los circuitos integrados de una computadora solo pueden reconocer y ejecutar un conjunto muy limitado de operaciones básicas.

A partir de esta idea, resulta natural preguntarse cómo es posible construir sistemas tan complejos como los que usamos hoy en día. La respuesta está en la forma en que estas operaciones simples se organizan y combinan.

En primer término, se comenzará por estudiar el concepto de **Arquitectura del Computador**. De manera intuitiva, la arquitectura de una computadora describe cómo está “armada” internamente y cómo funcionan sus distintas partes en conjunto. Esto incluye, por ejemplo, el procesador, la memoria y los mecanismos mediante los cuales se ejecutan los programas.

Comprender la arquitectura permite entender qué puede hacer una computadora, cómo lo hace y cuáles son sus limitaciones. Este conocimiento es fundamental para interpretar el funcionamiento de los programas y, más adelante, para escribir código más eficiente y cercano al hardware.

2. Concepto de Arquitectura del Computador

“La arquitectura del computador es el punto de encuentro entre la lógica del software y las limitaciones del hardware.”

Para comprender el funcionamiento de un sistema informático electrónico, es útil analizarlo desde distintos niveles de abstracción. Cada nivel permite enfocar la atención en ciertos aspectos específicos del sistema, ocultando temporalmente los detalles de los niveles inferiores. Desde el software que utiliza un usuario hasta los circuitos electrónicos que procesan las señales, estos niveles abarcan desde el nivel de aplicación, pasando por el sistema operativo, la arquitectura del conjunto de instrucciones, la microarquitectura y, finalmente, el nivel físico. Esta organización en capas facilita el diseño, análisis y comprensión de sistemas complejos, permitiendo que especialistas en distintas áreas puedan trabajar de manera coordinada. La Figura 1 muestra los niveles de abstracción mencionados.

El nivel de la **arquitectura del computador** define el comportamiento visible del hardware desde la perspectiva del programador. En este nivel se especifican las instrucciones disponibles, los modos de direccionamiento, los registros, el formato de los datos y la forma en que se manejan las interrupciones, entre otros aspectos. En resumen, este nivel de abstracción describe cómo se presenta la computadora a quien la programa.

La arquitectura x86, utilizada por los microprocesadores en la mayoría de las computadoras personales, se define mediante un conjunto de instrucciones y registros (memoria utilizada para almacenar temporalmente variables) disponibles para el programador.

Al abordar una nueva arquitectura, algunas preguntas clave son:

- ¿Cómo se representan los datos y las instrucciones?

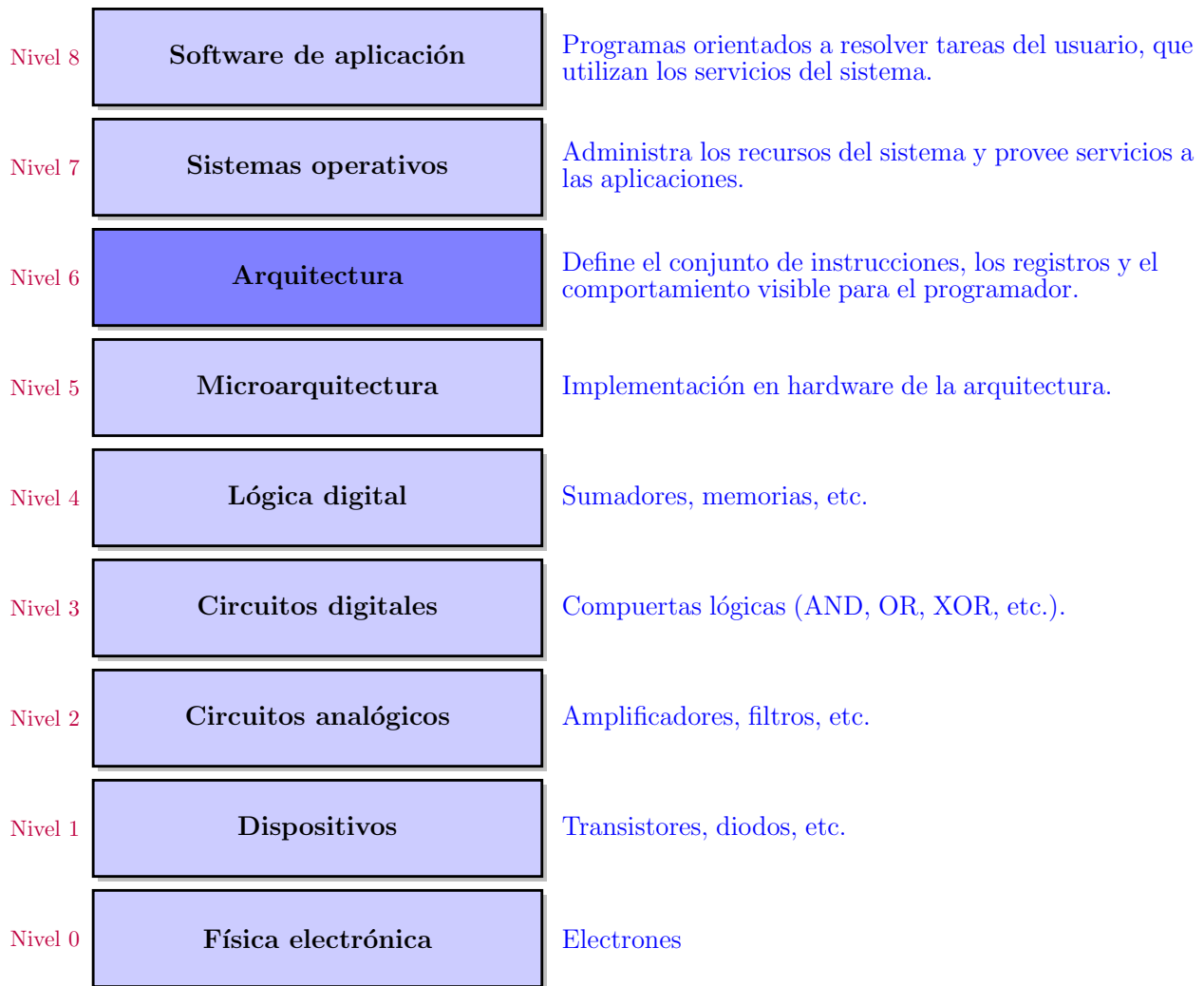


Figura 1: Niveles de abstracción de una computadora.

- ¿Cuál es la longitud de la palabra de datos?
- ¿Qué registros posee y cuál es su tamaño?
- ¿Cómo está organizada la memoria?
- ¿Cómo se direcciona y se accede a la memoria?
- ¿Qué instrucciones están disponibles?

A lo largo de la asignatura, iremos profundizando en los conceptos clave de Arquitectura del Computador para comprender su funcionamiento y diseño.

3. Arquitectura de von Neumann

“La computadora no distingue entre programa y dato: todo son bits en memoria.”

En 1946, el matemático y físico John von Neumann y sus colegas comenzaron el diseño de una nueva computadora, conocida como la computadora IAS (por sus siglas en inglés de *Institute for Advanced Study*), en el Instituto de Estudios Avanzados de Princeton. Aunque la computadora IAS no se completó hasta 1952, es el prototipo de todas las computadoras de propósito general posteriores. Un enfoque de diseño fundamental implementado en la computadora IAS es conocido como el concepto de **programa almacenado**.

El concepto de **programa almacenado** refiere a una organización en la cual las instrucciones que componen un programa se encuentran codificadas como secuencias de bits y almacenadas en memoria, desde donde el procesador las recupera y ejecuta automáticamente. De este modo, una misma máquina puede realizar distintas tareas simplemente cargando un programa diferente en memoria, sin necesidad de modificar su hardware.

La Figura 2 muestra la estructura simplificada de la **arquitectura von Neumann**:

- Una memoria principal, que almacena tanto datos como instrucciones.
- Una unidad aritmética-lógica (ALU) capaz de operar con datos binarios.
- Una unidad de control, que interpreta las instrucciones en la memoria y las ejecuta.
- Equipos de entrada/salida (I/O) operados por la unidad de control.

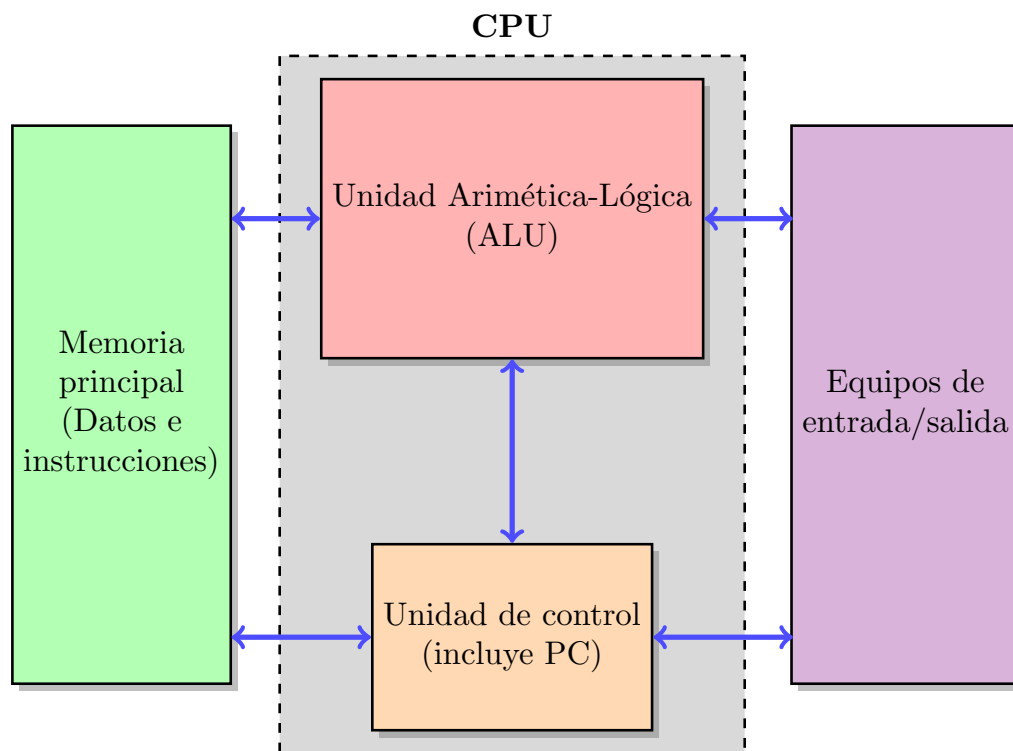


Figura 2: Estructura de una computadora con arquitectura von Neumann, donde una única memoria almacena tanto datos como instrucciones.

En este modelo, tanto las instrucciones como los datos residen en una única memoria (**almacenamiento unificado**) y son accedidos mediante los mismos mecanismos. En implementaciones modernas, aunque el modelo lógico es de memoria unificada, es común utilizar memorias caché separadas para instrucciones y datos dentro de la CPU (Unidad Central de Procesamiento).

La ejecución de un programa consiste en la repetición de un ciclo de instrucciones, en el cual la unidad de control obtiene una instrucción desde memoria (utilizando el contador de programa, PC), la interpreta y coordina su ejecución.

En un sistema típico, los programas y datos están almacenados inicialmente en almacenamiento secundario (por ejemplo, un disco rígido o de estado sólido) y se copian a la memoria principal antes de su ejecución. La CPU ejecuta el programa desde la memoria principal, repitiendo el ciclo de instrucciones.

Con pocas excepciones, las computadoras actuales siguen este esquema general, por lo que se las denomina **arquitecturas de von Neumann**.

La principal ventaja de esta arquitectura es su simplicidad y flexibilidad, derivadas del uso de una única memoria para almacenar tanto instrucciones como datos, lo que simplifica el diseño del hardware y permite tratar a los programas como datos. Sin embargo, el uso de una memoria compartida introduce una limitación conocida como el “*cuello de botella*” de von Neumann, debida a la competencia por el acceso a memoria entre instrucciones y datos. Este inconveniente se analizará en detalle en la Sección 6.

Una alternativa al modelo de von Neumann es la **arquitectura Harvard**, en la cual las instrucciones y los datos se almacenan en memorias físicamente separadas y se accede a ellas mediante buses independientes.

Esta diferencia estructural permite que la CPU pueda, en principio, acceder simultáneamente a instrucciones y datos, evitando la competencia por el acceso a memoria característica de la arquitectura de von Neumann.

Sin embargo, la arquitectura Harvard requiere mayor complejidad de hardware y menor flexibilidad, ya que separa estrictamente el espacio de direcciones de instrucciones y datos.

En la práctica, muchas arquitecturas modernas adoptan esquemas intermedios (a veces denominados *Harvard modificadas*), donde existe una memoria unificada a nivel lógico, pero se emplean memorias caché separadas para instrucciones y datos, combinando así ventajas de ambos enfoques. Por ejemplo, muchos procesadores actuales (incluyendo x86-64 y ARM) presentan una organización de memoria de tipo von Neumann a nivel del modelo de programación, pero utilizan cachés separadas de instrucciones y datos internamente.

4. Modelo de ejecución

Se puede modelar a la computadora como una máquina que ejecuta instrucciones sobre un estado bien definido. Este estado está compuesto por:

- un conjunto de registros ($R_1, R_2, \dots, R_n$), que son pequeñas unidades de memoria ubicadas en la CPU,
- una memoria principal direccionable ($M[0], M[1], \dots, M[n-1]$),
- un contador de programa (*program counter*, PC), que indica la dirección de la próxima instrucción a ejecutar.

Cada instrucción se interpreta como una transformación del estado de la máquina. En particular, puede leer y modificar registros, acceder a memoria y actualizar el PC.

La ejecución de un programa se describe como una secuencia de estas transformaciones, organizada mediante el ciclo de instrucción **fetch-decode-execute**:

1. **Fetch**: se obtiene de memoria la instrucción apuntada por el PC,
2. **Decode**: se interpreta la instrucción,
3. **Execute**: se realiza la operación y se actualiza el estado.

Ejemplo

Considérese la siguiente secuencia de pseudo-instrucciones:

Dirección de memoria	Instrucción
0	$R1 \leftarrow 5$
1	$R1 \leftarrow R1 + 3$
2	$M[100] \leftarrow R1$
3	$R2 \leftarrow M[100]$

Supongamos el estado inicial: $R1 = 0$, $R2 = 0$, $M[100] = 0$, $PC = 0$.

Paso 1

- Fetch: se obtiene la instrucción en la dirección $PC = 0$,
- Decode: se interpreta como “asignar 5 a $R1$ ”,
- Execute: se actualiza $R1 = 5$ y $PC = 1$.

Paso 2

- Fetch: se obtiene la instrucción en la dirección $PC = 1$,
- Decode: se interpreta como “sumar 3 a $R1$ ”,
- Execute: se actualiza $R1 = 8$ y $PC = 2$.

Paso 3

- Fetch: se obtiene la instrucción en la dirección $PC = 2$,
- Decode: se interpreta como “almacenar el valor de $R1$ en la dirección 100 de memoria”,
- Execute: se actualiza $M[100] = 8$ y $PC = 3$.

Paso 4

- Fetch: se obtiene la instrucción en la dirección $PC = 3$,
- Decode: se interpreta como “cargar en $R2$ el valor almacenado en la dirección 100”,
- Execute: se actualiza $R2 = 8$ y $PC = 4$.

De este modo, el ciclo fetch–decode–execute describe cómo, en cada paso, la máquina obtiene una instrucción, la interpreta y actualiza su estado. El valor del PC determina la secuencia de ejecución.

Relación con el modelo de Máquina de Turing: El modelo de ejecución presentado puede interpretarse como una instancia práctica de un esquema más general de cómputo basado en estados y transiciones. En particular, una Máquina de Turing también se define a partir de:

- un estado finito de control,
- una estructura de memoria (cinta),

- un conjunto de reglas de transición que actualizan el estado y la memoria.

En este sentido, la memoria principal puede verse como una generalización de la cinta, mientras que el contador de programa y los registros constituyen el estado finito de control. Cada instrucción del programa puede interpretarse como una regla de transición que transforma el estado de la máquina.

Esta correspondencia es conceptual: las computadoras reales implementan este esquema mediante arquitecturas concretas como von Neumann, donde las instrucciones son almacenadas en memoria y ejecutadas secuencialmente.

5. Organización básica de una computadora

Los componentes básicos de una computadora incluyen una Unidad Central de Procesamiento (CPU), almacenamiento primario o memoria de acceso aleatorio (RAM, por su sigla en inglés *Random Access Memory*), almacenamiento secundario (disco duro, disco de estado sólido, memoria USB, etc.), dispositivos de entrada/salida (pantalla, teclado, mouse, etc.) y una interconexión entre los diferentes componentes denominada buses.

En términos generales, la CPU es responsable de ejecutar instrucciones, la memoria principal almacena los datos y programas en ejecución, el almacenamiento secundario permite conservar información de manera persistente, y los dispositivos de entrada/salida posibilitan la interacción con el entorno. Los buses permiten la transferencia de información entre estos componentes.

Un diagrama muy básico de la organización de una computadora se muestra en la Figura 3.

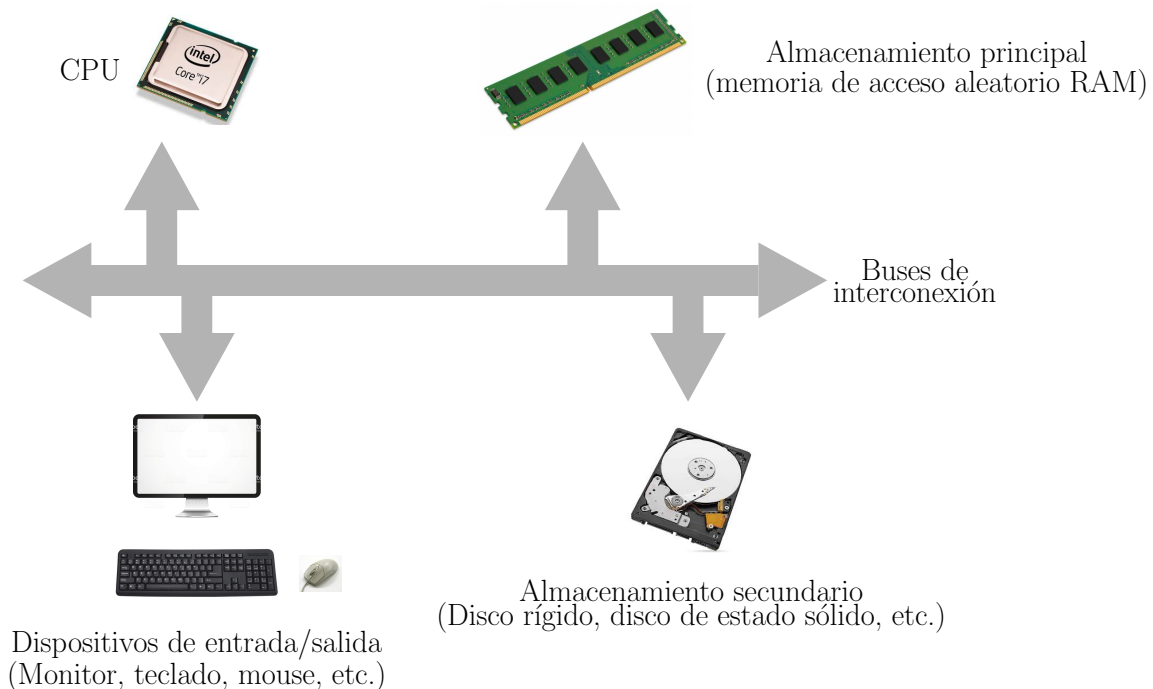


Figura 3: Componentes básicos de una computadora.

5.1. Unidad Central de Procesamiento (CPU)

“La CPU no piensa: ejecuta. Y sin embargo, todo lo que parece inteligencia pasa inevitablemente por ella.”

La unidad central de procesamiento (CPU), o simplemente el procesador, es el componente encargado de ejecutar las instrucciones almacenadas en la memoria principal. La Figura 4 muestra de manera esquemática una CPU con sus principales componentes.

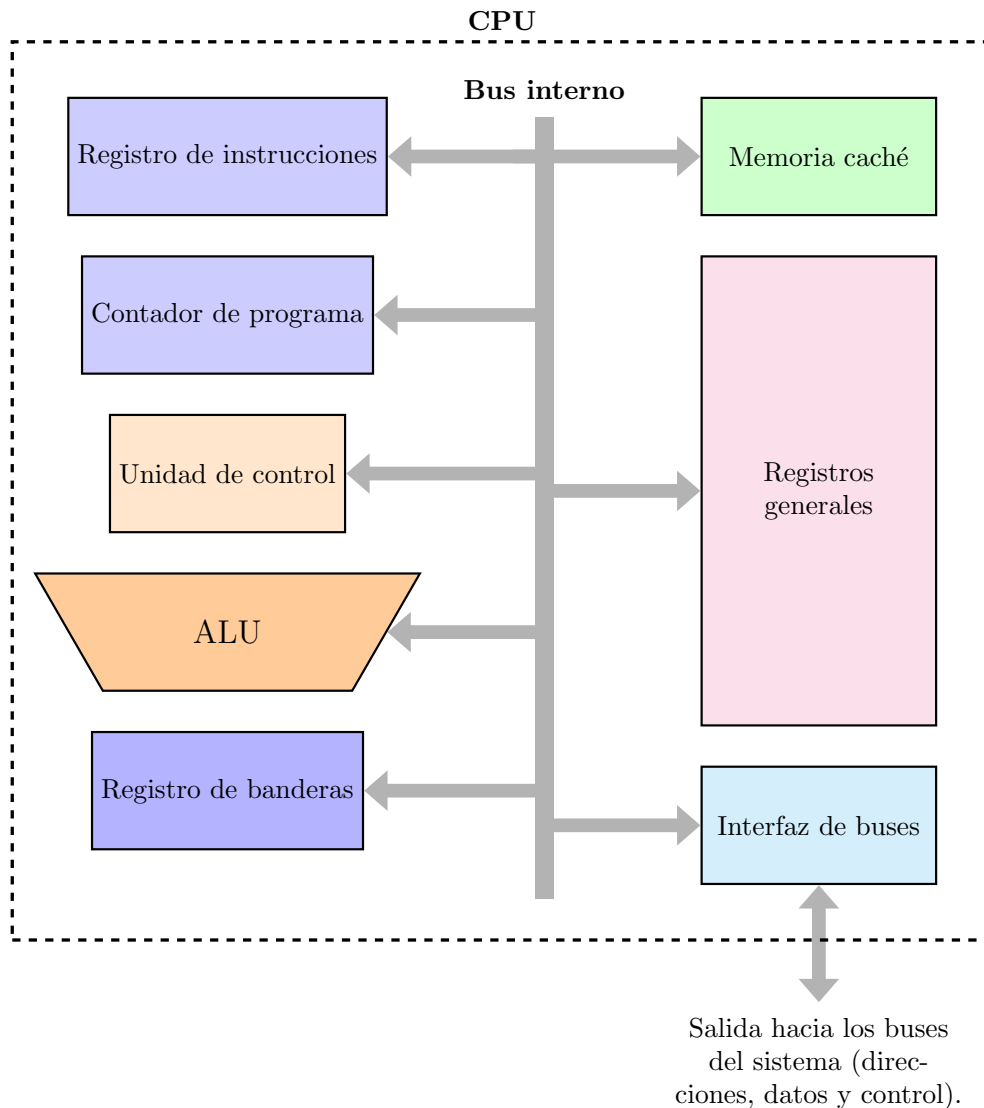


Figura 4: Esquema simplificado de la organización interna de una CPU.

Estos son algunos ejemplos de operaciones simples que la CPU puede realizar a partir de la ejecución de una instrucción:

- **Cargar:** copiar información desde la memoria principal a un registro, sobrescribiendo su contenido previo.
- **Almacenar:** copiar información desde un registro hacia una ubicación de la memoria principal, sobrescribiendo su contenido previo.
- **Operar:** realizar una operación aritmética o lógica sobre datos almacenados en registros y guardar el resultado en un registro.
- **Transferir el control:** modificar la secuencia de ejecución del programa alterando el contenido del contador de programa (PC).

5.2. Registros

“Los registros no almacenan mucho, pero almacenan lo importante.”

El intercambio de datos entre la CPU y la memoria es fundamental en una arquitectura von Neumann: las instrucciones y los operandos deben obtenerse de la memoria, y algunas instrucciones también almacenan resultados en ella. Este proceso puede generar un cuello de botella debido a la diferencia de velocidades entre la CPU y la memoria, lo que hace que la CPU pierda tiempo mientras espera la respuesta del chip de memoria. Para mitigar este problema, el procesador cuenta con registros, conjunto de celdas de memoria integradas que, aunque limitadas en cantidad, son extremadamente rápidas y mejoran significativamente el rendimiento del procesamiento.

Un **registro** es una memoria de alta velocidad y baja capacidad, integrada en el microprocesador, que permite almacenar temporalmente y acceder a valores frecuentemente utilizados, especialmente en operaciones matemáticas.

La Figura 5 muestra un diagrama esquemático de un registro compuesto por n celdas de memoria. Cada celda consiste en un circuito electrónico biestable, conocido como *flip-flop*, que puede adoptar uno de dos estados posibles (0 o 1), lo que permite almacenar un bit de información.

Un **bit** (acrónimo de *binary digit*) es la unidad mínima de información en computación y sistemas digitales. Puede representar dos estados posibles, típicamente denotados como 0 y 1.

El **sistema binario** puede entenderse como una traducción directa de la física a la lógica: dos estados claramente distinguibles, como presencia o ausencia de señal, o niveles alto y bajo de energía. En este sentido, las computadoras no “comprenden” números en términos abstractos, sino que detectan y manipulan niveles físicos de energía. Los valores 0 y 1 constituyen entonces una abstracción de estos fenómenos eléctricos, elegida por su simplicidad y robustez. La confiabilidad del hardware se apoya en la capacidad de diferenciar inequívocamente entre estos dos estados, incluso en presencia de perturbaciones. Dado que el ruido es inevitable en cualquier sistema físico, el uso de una representación binaria permite tolerarlo y minimizar su impacto en la información procesada.

El concepto de bit y su uso en el sistema de representación binario se abordan en detalle en el Apunte **Representación Computacional de Datos**.

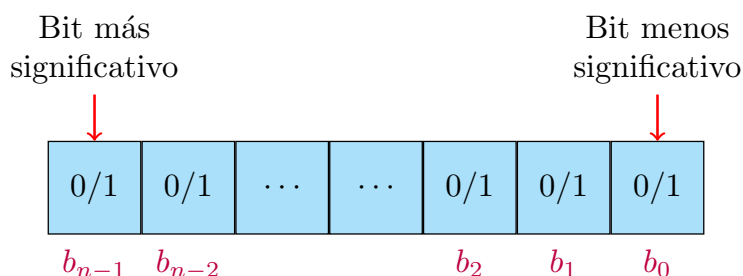


Figura 5: Esquema de un registro de n bits.

Las principales características de los registros son las siguientes:

- **Velocidad:** Son extremadamente rápidos en comparación con la memoria principal (RAM), ya que están integrados directamente en el procesador.

- **Capacidad:** Tienen una capacidad limitada, típicamente de 8, 16, 32 o 64 bits, según la arquitectura.
- **Cantidad:** Cantidad limitada de registros, determinada por la arquitectura.
- **Función:** Se utilizan para almacenar datos temporales, como resultados de operaciones aritméticas, direcciones de memoria o valores intermedios durante la ejecución de un programa.

5.3. Contador de programa

“El Program Counter es la brújula que guía la ejecución de un programa.”

El **contador de programa** (conocido en inglés como *Program Counter* o PC) es un registro especial en la CPU que contiene la dirección de memoria de la siguiente instrucción que debe ser ejecutada. Es uno de los registros más importantes en la arquitectura del computador, ya que controla la secuencia de ejecución de las instrucciones. En todo momento, el PC apunta a una instrucción en lenguaje máquina almacenada en la memoria principal. Desde que el sistema se enciende hasta que se apaga, el procesador ejecuta repetidamente la instrucción señalada por el contador de programa y actualiza el PC para que apunte a la siguiente instrucción.

El funcionamiento de un procesador se puede interpretar mediante un modelo de ejecución de instrucciones muy simple. En este modelo, las instrucciones se ejecutan en secuencia estricta, y la ejecución de una sola instrucción implica una serie de pasos: el procesador lee la instrucción de la memoria a la que apunta el contador de programa (PC), interpreta los bits de la instrucción, realiza la operación simple dictada por la instrucción y luego actualiza el PC para que apunte a la siguiente instrucción. Esta siguiente instrucción puede estar o no contigua en memoria a la instrucción que acaba de ejecutarse.

5.4. Unidad Aritmética Lógica (ALU)

“Toda computación, por compleja que parezca, termina reduciéndose a operaciones de la ALU.”

La CPU incluye varias unidades funcionales, entre ellas la **unidad aritmética lógica** (ALU, por su sigla en inglés *Arithmetic Logic Unit*), que es la parte del procesador encargada de realizar los cálculos aritméticos y lógicos. Su función es generar nuevos datos y valores de dirección.

A nivel de la ALU solo están disponibles algunas pocas operaciones muy simples:

- Operaciones aritméticas entre operandos,
- Operaciones lógicas de bits,
- Operaciones de desplazamiento de bits.

La Figura 6 presenta un esquema de la ALU, cuyas entradas incluyen dos operandos y una señal de control que determina la operación a ejecutar. Las salidas comprenden el resultado de la operación y las banderas asociadas.

Por ejemplo, si la señal de operación indica una suma, la ALU suma los operandos A y B, y el resultado se coloca en la salida junto con las banderas correspondientes.

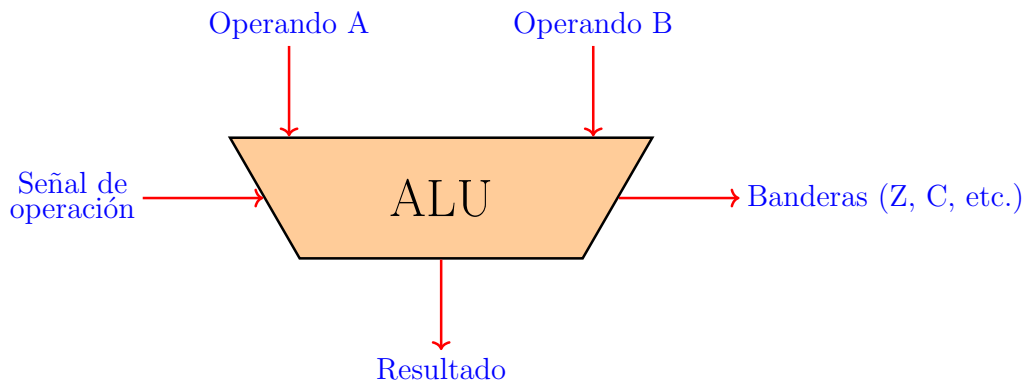


Figura 6: Esquema de la unidad aritmética lógica (ALU).

Ejemplo

Es interesante analizar, aunque sea a modo de ejemplo, cómo funciona la ALU, en particular en la realización de la operación de suma. Esta operación se implementa mediante compuertas lógicas, con las cuales pueden construirse sumadores que permiten sumar bit a bit los operandos. Cada sumador se encarga de sumar el n -ésimo bit de cada operando junto con el acarreo proveniente de la etapa anterior (c_{in}). Como resultado, se obtiene el bit de suma y un nuevo acarreo (c_{out}), que se utiliza como entrada en la siguiente etapa del sumador.

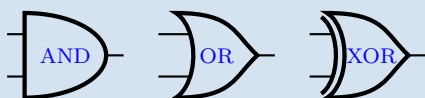
A continuación vemos la tabla de la verdad de dicha operación para el bit n -ésimo:

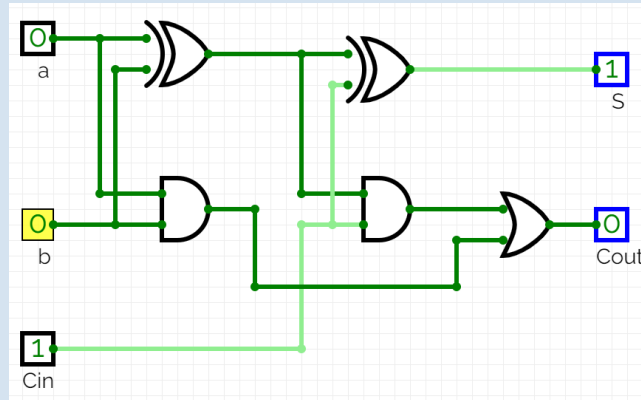
a	b	c_{in}	s	c_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

A partir de la tabla anterior podemos llegar a la siguiente expresión lógica:

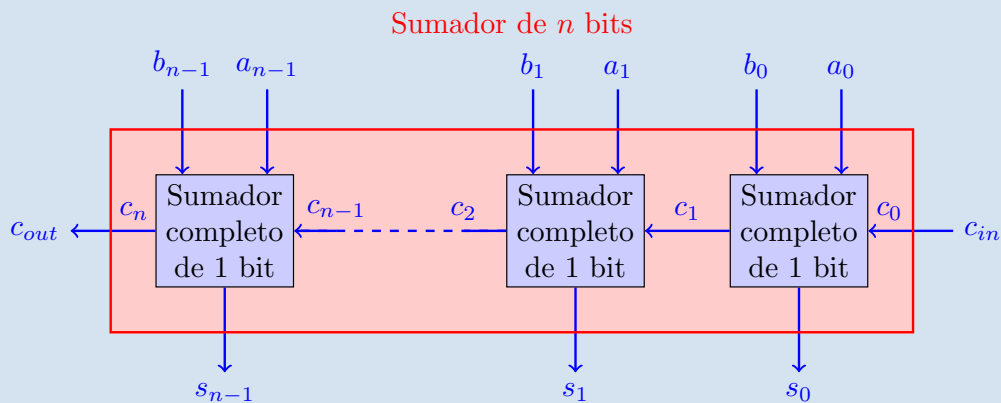
$$\begin{aligned}
 s &= (a \oplus b) \oplus c_{in} \\
 c_{out} &= ab + c_{in}(a \oplus b)
 \end{aligned}$$

Podemos representar esta expresión como un circuito lógico. A continuación se presentan algunos de los símbolos de las compuertas lógicas más comunes que se utilizan en los diagramas de circuitos digitales:





Este circuito se conoce como “sumador completo” (*full-adder*). Para sumar dos números de n bits, es necesario conectar en cascada n sumadores completos, como se observa en la siguiente figura:



De manera análoga, la CPU implementa también operaciones lógicas, aritméticas, desplazamientos y rotaciones.

5.5. Buses

En todo el sistema existe un conjunto de conductores eléctricos, denominado **buses**, que transporta información entre los distintos componentes. Estos buses suelen estar diseñados para transferir datos en unidades de tamaño fijo, conocidas como palabras. La Figura 7 muestra el modelo de buses de una computadora.

- **Bus de Datos:** indica qué información se está transmitiendo.
- **Bus de Dirección:** señala hacia dónde se transfiere la información.
- **Bus de Control:** especifica cómo se transfiere la información y qué se hará con ella.
- La CPU genera direcciones que se transmiten a través del bus de direcciones.
- La memoria recibe esas direcciones mediante el mismo bus.
- La memoria nunca genera direcciones, y la CPU nunca las recibe.

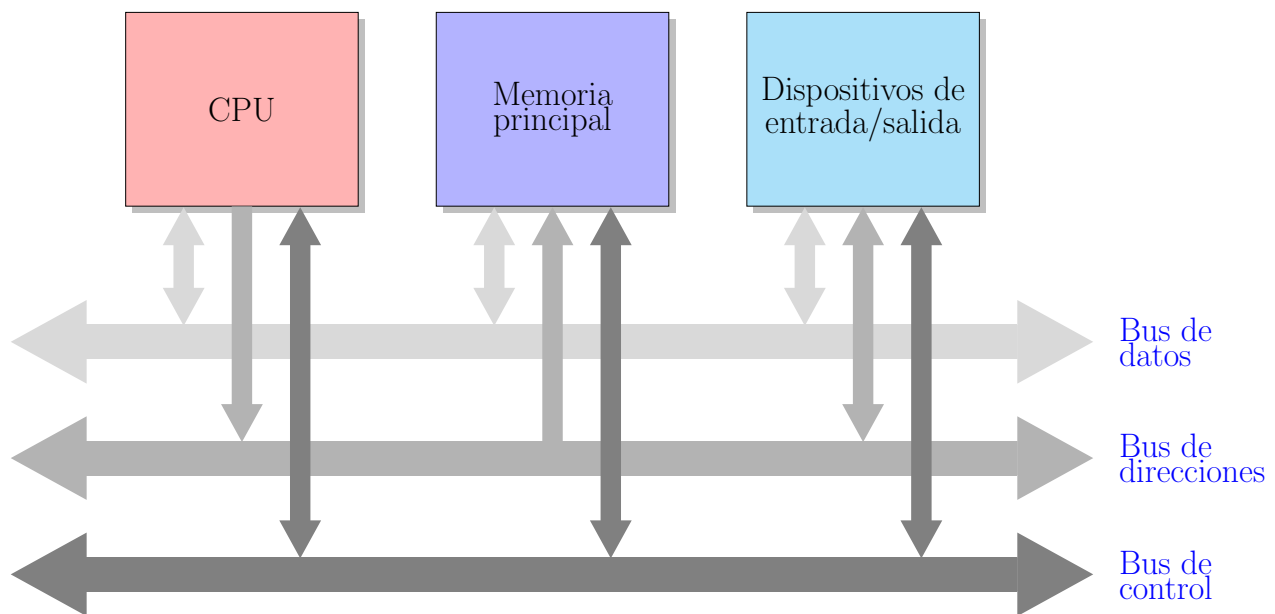


Figura 7: Modelo de buses del sistema.

El número de bytes¹ en una palabra (es decir, el tamaño de palabra) es un parámetro fundamental del sistema que varía entre distintas arquitecturas y repercute directamente en el ancho de los buses, entendido como la cantidad de bits que pueden transportar simultáneamente. La mayoría de las computadoras actuales utilizan tamaños de palabra de 4 bytes (32 bits) u 8 bytes (64 bits). Este tema se desarrolla con mayor detalle a lo largo de la asignatura.

5.6. Memoria principal

“La memoria principal vive en el presente: guarda lo necesario, pero olvida todo al final.”

La **memoria principal** es un dispositivo de almacenamiento temporal que contiene tanto el programa como los datos que se están manipulando mientras el procesador ejecuta el programa. Esta característica es fundamental en las computadoras basadas en el concepto de *Máquina de Von Neumann*. Físicamente, la memoria principal está constituida por una colección de chips de memoria dinámica de acceso aleatorio (DRAM, por sus siglas en inglés *Dynamic Random Access Memory*). A nivel lógico, la memoria está organizada como una matriz lineal de bytes, cada uno con su propia dirección única (dirección de memoria).

La memoria principal se caracteriza por ofrecer tiempos de acceso muy reducidos en comparación con los dispositivos de almacenamiento permanente, como los discos rígidos, lo que permite a la CPU leer y escribir datos con gran rapidez. Sin embargo, esta velocidad tiene un costo: se trata de una memoria *volátil*, es decir, su contenido se pierde cuando se interrumpe el suministro de energía. Por esta razón, la memoria principal actúa como un espacio de trabajo temporal, almacenando únicamente la información necesaria para la ejecución de los programas en curso.

5.7. Memoria Caché

“La memoria caché es el arte de tener a mano lo que todavía no se pidió.”

¹Un byte es un conjunto ordenado de ocho bits. A su vez, un bit (acrónimo de *binary digit*) es la unidad mínima de información en un sistema binario. Un bit puede representar uno de dos valores: 0 o 1. Este tema se explorará con mayor detalle en el apunte **Representación Computacional de Datos**.

La **memoria caché** es un pequeño subconjunto del almacenamiento primario que se encuentra integrado en el chip de la CPU. Está diseñada para combinar la alta velocidad de acceso de los registros de la CPU, que son costosos y rápidos, con el gran tamaño de la memoria principal, que es menos costosa pero más lenta. De este modo, se establece una jerarquía en la que la memoria principal es relativamente grande y lenta, mientras que la memoria caché es más pequeña pero mucho más rápida.

La memoria caché contiene copias de porciones de la memoria principal. Cuando el procesador intenta leer una palabra de memoria, primero verifica si esa palabra está en la caché. Si es así, la palabra se entrega al procesador de inmediato. Si no está en la caché, se lee un bloque de la memoria principal, que consta de un número fijo de palabras, y se carga en la caché antes de entregar la palabra solicitada al procesador. Este proceso se beneficia del fenómeno de localidad, que sugiere que cuando se carga un bloque de datos en la caché para satisfacer una referencia, es probable que se realicen referencias futuras a esa misma ubicación o a otras palabras dentro del mismo bloque².

La memoria caché tiene su propia jerarquía, que se organiza en tres niveles:

- **Caché L1 (Nivel 1):** Es la memoria caché más cercana al núcleo del procesador y, por lo tanto, la más rápida. Suele estar dividida en dos partes: una para instrucciones y otra para datos. Debido a su proximidad y alta velocidad, la caché L1 es relativamente pequeña, normalmente entre 16 KB y 64 KB por núcleo.
- **Caché L2 (Nivel 2):** Se encuentra un nivel por debajo de la caché L1 y es más grande, generalmente entre 256 KB y 1 MB por núcleo. Aunque es más lenta que la L1, sigue siendo mucho más rápida que la memoria principal (RAM). La caché L2 puede estar compartida entre varios núcleos o ser exclusiva para cada uno, dependiendo del diseño del procesador.
- **Caché L3 (Nivel 3):** Es el nivel más grande y lento de la jerarquía de caché. Suele estar compartida por todos los núcleos del procesador y su tamaño varía, pudiendo llegar a varios megabytes (de 4 MB a 64 MB o más). Aunque es más lenta que las cachés L1 y L2, la L3 sigue siendo más rápida que acceder a la RAM, por lo que juega un papel crucial en la mejora del rendimiento del sistema.

Cada nivel de caché actúa como un intermediario para el siguiente, con el objetivo de reducir los tiempos de acceso a los datos y mejorar la eficiencia general del procesamiento. La Figura 8 muestra esquemáticamente la jerarquía de una memoria caché típica en una CPU de dos núcleos.

5.8. Memoria secundaria

“Lo que la RAM olvida al apagar, la memoria secundaria lo guarda para siempre.”

El principal problema de las memorias RAM es su volatilidad. Esto significa que la memoria RAM pierde toda la información almacenada cuando la computadora se apaga o se reinicia. A diferencia de los dispositivos de almacenamiento no volátiles, como los discos duros o las SSD, que conservan los datos incluso cuando la energía se desconecta, la memoria RAM solo retiene los datos mientras está alimentada. Esta característica limita su uso a tareas que requieren acceso rápido a datos temporales durante el funcionamiento de la computadora, pero no es adecuada para el almacenamiento permanente de información.

²Los registros de la CPU y la memoria caché se describirán con mayor detalle en la Sección 6 y otros apuntes. Sin embargo, cabe señalar que el diseño interno y la configuración de un procesador moderno son sumamente complejos, y esta asignatura proporciona un enfoque de alto nivel muy simplificado de algunas unidades funcionales clave dentro de una CPU.

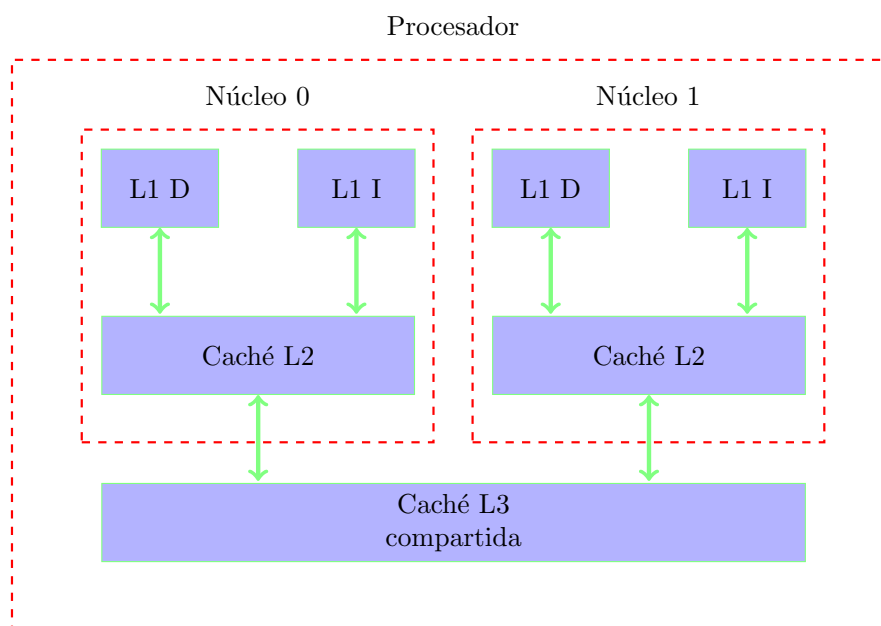


Figura 8: Esquema jerárquico de la memoria caché en una CPU de dos núcleos.

Un disco duro es un dispositivo de almacenamiento de datos magnético que se utiliza en las computadoras para guardar permanentemente archivos y programas. Una de las principales diferencias entre el disco duro y la memoria RAM es la velocidad. La memoria RAM proporciona un acceso rápido y aleatorio a los datos y programas, lo que permite al procesador acceder a ellos de manera eficiente. En comparación, los discos duros son significativamente más lentos en términos de velocidad de acceso, ya que el brazo de lectura/escritura debe moverse físicamente para acceder a los datos almacenados en los platos giratorios.

Un SSD (Solid State Drive) es un dispositivo de almacenamiento de datos que utiliza memoria flash para guardar permanentemente archivos y programas en la computadora. A diferencia de los discos duros tradicionales, que emplean platos magnéticos giratorios y cabezales de lectura/escritura, una SSD no tiene partes móviles y utiliza chips de memoria flash para almacenar y acceder a los datos.

La tecnología SSD ofrece un rendimiento superior en comparación con los discos duros debido a la ausencia de partes mecánicas móviles. Los datos se almacenan en chips de memoria flash, que permiten un acceso y lectura mucho más rápidos. Como resultado, las SSD proporcionan un mejor rendimiento en términos de velocidad de lectura/escritura y tiempo de acceso.

Observación

Los dispositivos de memoria no volátiles se denominan **memoria secundaria** porque no son accedidos directamente por la CPU durante la ejecución, sino que se utilizan como almacenamiento persistente cuyos datos deben transferirse a la memoria principal para su procesamiento.

5.9. Dispositivos de entrada/salida

Los **dispositivos de entrada/salida** (I/O *devices*) son la conexión del sistema con el mundo externo. La computadora del ejemplo en la Figura 3 tiene cuatro dispositivos de E/S: un teclado y un mouse para la entrada del usuario, una pantalla para la salida, y una unidad de disco (o disco rígido) para el almacenamiento a largo plazo de datos y programas.

Cada dispositivo de E/S está conectado al bus de E/S mediante un controlador o un adaptador. Los controladores son conjuntos de chips integrados en el propio dispositivo o en la placa

de circuito impreso principal del sistema, conocida comúnmente como placa base. Un adaptador, en cambio, es una tarjeta que se conecta a una ranura en la placa base. En ambos casos, el propósito es transferir información de ida y vuelta entre el bus de E/S y un dispositivo de E/S.

6. Jerarquía de memoria

“Idealmente, se desearía una capacidad de memoria indefinidamente grande de manera que cualquier palabra en particular estuviera disponible de inmediato. Sin embargo, nos vemos obligados a reconocer la posibilidad de construir una jerarquía de memorias, cada una de las cuales tiene una capacidad mayor que la anterior, pero que es menos rápidamente accesible.”

A. W. Burks, H. H. Goldstine, y J. von Neumann

Discusión preliminar sobre el diseño lógico de un instrumento de cálculo electrónico, 1946.

El desarrollo de los procesadores ha permitido incrementar significativamente su capacidad de cómputo, en línea con la tendencia descrita por la **Ley de Moore**. Sin embargo, la velocidad de acceso a la memoria no ha aumentado al mismo ritmo, generando una creciente brecha de rendimiento entre el procesador y la memoria, conocida como *Memory Wall*.

La **Ley de Moore** es una observación empírica formulada por Gordon Moore en 1965, según la cual la cantidad de transistores que puede integrarse en un circuito integrado tiende a duplicarse aproximadamente cada dos años. Esta tendencia permitió un crecimiento sostenido de la capacidad de cómputo de los procesadores.

Para reducir el impacto de esta brecha, los dispositivos de almacenamiento en un sistema informático se organizan en una **jerarquía de memoria**, como se muestra en la Figura 9. El objetivo es maximizar el rendimiento manteniendo los datos más utilizados lo más cerca posible del procesador.

A medida que se desciende en la jerarquía, los dispositivos se vuelven más lentos, tienen mayor capacidad y resultan menos costosos por byte. En cambio, en los niveles superiores se encuentran dispositivos más rápidos y con mayor costo.

En la cúspide de la jerarquía se ubican los registros, seguidos por la memoria caché, ambos integrados en la CPU. A continuación se encuentra la memoria principal y, en los niveles inferiores, los dispositivos de almacenamiento masivo, como discos rígidos y cintas magnéticas.

El principio fundamental de esta jerarquía es que cada nivel actúa, en términos generales, como un *buffer* del nivel inferior, almacenando temporalmente un subconjunto de sus datos para reducir los tiempos de acceso. De este modo, los registros funcionan como *buffer* de la caché L1; la memoria caché, de la memoria principal; y esta, a su vez, de los dispositivos de almacenamiento secundario.

7. Clasificación de arquitecturas

“La complejidad puede estar en el hardware o en el software, pero nunca desaparece.”

Las arquitecturas **CISC** (Complex Instruction Set Computer) y **RISC** (Reduced Instruction Set Computer) representan dos enfoques distintos en el diseño de procesadores. La principal diferencia radica en la complejidad y la cantidad de instrucciones que cada una ofrece. El término **conjunto de instrucciones de la arquitectura** (ISA, por sus siglas en inglés

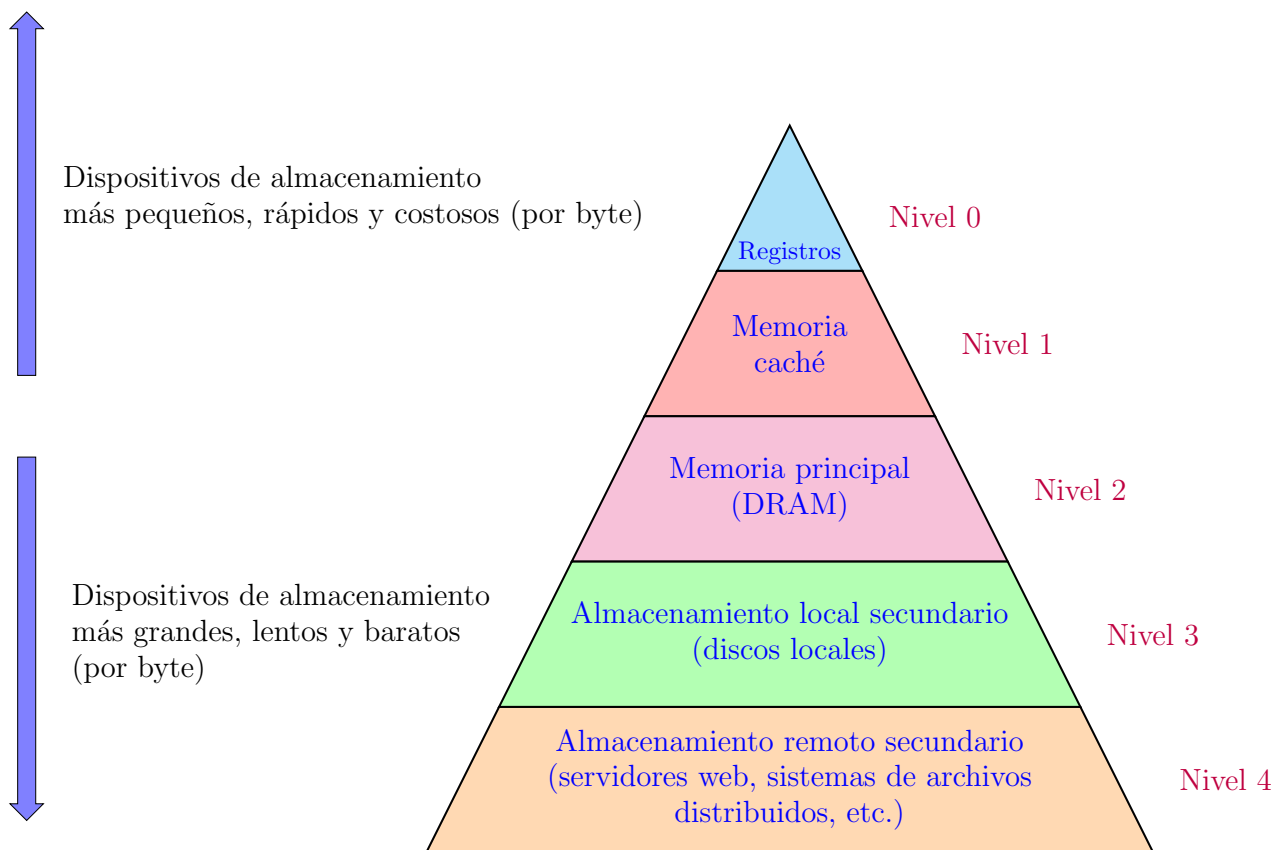


Figura 9: Diagrama piramidal de la jerarquía de memoria.

Instruction Set Architecture) hace referencia al conjunto real de instrucciones disponibles para controlar la CPU.

Una **instrucción** es una operación elemental codificada en binario que la CPU puede ejecutar, especificando la acción a realizar y los operandos involucrados.

El ISA actúa como una interfaz entre el hardware y el software, al especificar qué operaciones puede realizar el procesador y cómo se ejecutan. Define los tipos de datos admitidos, los registros disponibles, la forma en que el hardware gestiona la memoria principal, características clave como la memoria virtual, el conjunto de instrucciones ejecutables y el modelo de entrada/salida en distintas implementaciones. Además, el ISA puede ampliarse mediante la incorporación de nuevas instrucciones, funcionalidades adicionales o el soporte para direcciones y tamaños de datos más grandes.

Los procesadores CISC disponen de un conjunto amplio y variado de instrucciones, muchas de las cuales pueden ejecutar tareas complejas en una sola operación. Esto permite que los programas en lenguaje ensamblador sean más compactos, pero a costa de una mayor complejidad en el hardware y tiempos de ejecución variables.

En contraste, las arquitecturas RISC se basan en un conjunto reducido y uniforme de instrucciones, generalmente de ejecución rápida y duración fija. Esto simplifica el diseño del procesador y facilita la implementación de técnicas como la segmentación (pipelining), lo que mejora el rendimiento y la eficiencia energética. Además, en RISC, el acceso a memoria está limitado a instrucciones específicas de carga y almacenamiento, lo que favorece una ejecución más predecible.

Estas diferencias hacen que CISC sea más común en arquitecturas tradicionales como x86, mientras que RISC ha ganado protagonismo en dispositivos móviles y sistemas embebidos, como los basados en ARM, debido a su bajo consumo y alto rendimiento.

La Tabla 1 presenta una comparación entre las arquitecturas CISC (Complex Instruction Set Computer) y RISC (Reduced Instruction Set Computer). Se destacan sus principales diferencias en cuanto al diseño del conjunto de instrucciones, el acceso a memoria, la complejidad del hardware y del compilador, así como su eficiencia y ámbitos de aplicación típicos.

Tabla 1: Comparación entre arquitecturas CISC y RISC

Característica	CISC (Complex Instruction Set Computer)	RISC (Reduced Instruction Set Computer)
Conjunto de instrucciones	Grande y complejo	Reducido y simple
Tiempo de ejecución de instrucciones	Variable (dependiendo de la instrucción)	Fija (una instrucción por ciclo, típicamente)
Acceso a memoria	Las instrucciones en general pueden acceder a memoria directamente	Solo las instrucciones específicas de carga/almacenamiento acceden a memoria
Formato de instrucciones	Varios formatos y longitudes	Formato uniforme (tamaño fijo)
Uso de microcódigo	Frecuente (traducción interna a microinstrucciones)	Poco frecuente o inexistente
Tamaño del compilador	Más simple (mayor trabajo en hardware)	Más complejo (mayor trabajo en software)
Tamaño del hardware	Más grande y complejo	Más pequeño y eficiente
Tiempos de ejecución	Puede ser más lento por la complejidad de las instrucciones	Más rápido por simplicidad y paralelismo
Ejemplos típicos de procesadores	Intel x86	ARM, MIPS, RISC-V
Eficiencia energética	Menor eficiencia y mayor consumo	Mayor eficiencia y menor consumo
Aplicaciones típicas	Computadoras personales y servidores tradicionales	Dispositivos móviles y embebidos

En esta asignatura nos vamos a focalizar en primer lugar en la arquitectura x86-64. El x86-64 es un diseño de CPU de tipo CISC. Esto se refiere a la filosofía de diseño del procesador interno. Los procesadores CISC generalmente incluyen una amplia variedad de instrucciones (a veces superpuestas), diferentes tamaños de instrucciones y una amplia gama de modos de direccionamiento. El término se acuñó retroactivamente en contraste con las arquitecturas tipo RISC. Una de las arquitecturas tipo RISC es la denominada ARM, la cual estudiaremos al final del curso en el apunte *Conceptos básicos del ensamblador y arquitectura ARM*.

8. Microarquitectura

La **microarquitectura** es el diseño interno de un procesador que determina cómo se implementan físicamente las instrucciones definidas por la arquitectura del conjunto de instrucciones (ISA). Incluye componentes como unidades funcionales, registros, buses, cachés, decodificado-

res de instrucciones, unidades de ejecución y mecanismos de control de flujo y paralelismo. En particular, la arquitectura del computador especifica qué instrucciones existen y cómo deben comportarse, mientras que la microarquitectura determina cómo se ejecutan esas instrucciones en una implementación concreta. La Figura 1 muestra la ubicación de la microarquitectura dentro de los distintos niveles de abstracción de una computadora.

Una arquitectura particular, como la x86-64, puede tener muchas microarquitecturas diferentes (como Intel Haswell vs. Intel Skylake), cada una con diferentes rendimiento, costo y complejidad. Todas las microarquitecturas de una determinada arquitectura ejecutan los mismos programas, pero sus diseños internos pueden variar ampliamente. En esta asignatura nos centraremos en las arquitecturas y no abordaremos las microarquitecturas.

9. Breve historia de las computadoras

“La historia de las computadoras es la historia de cómo automatizar el pensamiento paso a paso.”

El desarrollo de las computadoras puede entenderse como una evolución progresiva de mecanismos y conceptos orientados a automatizar el procesamiento de información. Desde los primeros dispositivos mecánicos de cálculo hasta las arquitecturas electrónicas modernas, cada etapa ha estado impulsada por la búsqueda de mayor velocidad, precisión y flexibilidad. Más allá de los avances tecnológicos, esta evolución refleja también un cambio en las ideas fundamentales: el pasaje desde máquinas diseñadas para tareas específicas hacia sistemas de propósito general basados en el concepto de programa almacenado. En este sentido, la historia de la computación no constituye únicamente una sucesión de dispositivos, sino una transformación continua en la forma de representar, organizar y ejecutar procesos.

Desde sus orígenes se han desarrollado numerosos tipos de computadoras, algunos con un impacto decisivo y otros con una incidencia más limitada. El objetivo de esta sección es presentar una visión sintética de los inicios de la computación, destacando aquellos hitos que resultan fundamentales para comprender su evolución.

9.1. Computadoras mecánicas

- Blaise Pascal (1623–1662). En 1642 construyó un dispositivo completamente mecánico, basado en engranajes, que solo podía realizar sumas y restas.
- Gottfried von Leibniz (1646–1716). Construyó otra máquina completamente mecánica que, además de sumar y restar, podía multiplicar y dividir.
- Charles Babbage (1792–1871). Construyó una máquina diseñada para ejecutar un único algoritmo con el fin de calcular tablas numéricas útiles para la navegación. Los resultados eran perforados en una placa de cobre.
- Posteriormente, desarrolló la máquina analítica, capaz de ejecutar distintos algoritmos. Contaba con cuatro componentes: el almacén (memoria), el molino (unidad de cómputo), la sección de entrada (lector de tarjetas perforadas, ver Figura 11) y la sección de salida (resultados impresos o perforados). Podía sumar, restar, multiplicar y dividir operandos.
- Ada Lovelace (1815–1852). Es considerada la primera programadora de computadoras del mundo. Colaboró con Charles Babbage en la descripción de la máquina analítica y escribió el primer algoritmo destinado a ser ejecutado por una máquina, anticipando conceptos fundamentales de la programación moderna, como los bucles y las subrutinas. Su visión iba más allá del simple cálculo numérico: comprendió que la máquina podía

manipular símbolos y realizar tareas generales siguiendo instrucciones, lo que la llevó a imaginar aplicaciones más amplias, como la generación automática de música. A pesar de la originalidad y profundidad de sus ideas, su trabajo no fue valorado en su tiempo y solo comenzó a ser reconocido muchas décadas después.

- Konrad Zuse (1910–1995). Construyó una serie de máquinas calculadoras automáticas utilizando contactores electromagnéticos³.
- John Atanasoff (1903–1995). Diseñó una máquina que utilizaba aritmética binaria y una memoria basada en condensadores, empleando un proceso que denominó “refrescar la memoria”. Aunque el concepto era correcto, la máquina nunca llegó a funcionar por problemas de implementación.
- George Stibitz (1904–1995). Construyó una máquina más primitiva que la de Atanasoff, aunque sí llegó a funcionar.
- Howard Aiken (1900–1973). Construyó, utilizando relés, la máquina de propósito general que Babbage no había logrado construir con ruedas dentadas. Su primera máquina (Mark I) se completó en Harvard en 1944 (ver Figura 10). Contaba con 72 palabras de 23 dígitos decimales cada una y un tiempo de instrucción de 6 segundos. Las entradas y salidas se realizaban mediante cintas de papel perforadas (ver Figura 11). Para cuando se desarrolló la Mark II, las máquinas basadas en relés ya eran obsoletas.



Figura 10: Imagen de la computadora Mark I (fuente:[1]).

9.2. Computadoras con válvulas de vacío

La válvula electrónica —también conocida como válvula de vacío, tubo de vacío o bulbo— es un componente utilizado para amplificar, conmutar o modificar señales eléctricas mediante el control del movimiento de electrones en un espacio “vacío” a muy baja presión, o en presencia de gases especialmente seleccionados. Fue el componente clave que posibilitó el desarrollo de la

³También denominados relés o relevadores. Un relé es un dispositivo electromagnético que funciona como un interruptor controlado por un circuito eléctrico, en el que, por medio de una bobina y un electroimán, se acciona un juego de uno o varios contactos que permiten abrir o cerrar otros circuitos eléctricos independientes.

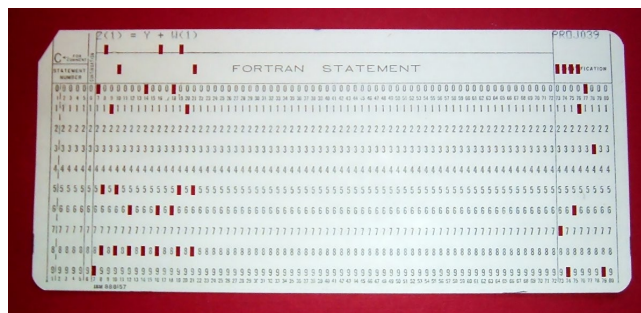


Figura 11: Imagen de una tarjeta perforada (Estándar de 80 columnas).

electrónica durante la primera mitad del siglo XX, impulsando el crecimiento y la comercialización de tecnologías como la radiodifusión, la televisión, el radar, el audio, las redes telefónicas, las computadoras analógicas y digitales, y los sistemas de control industrial. Si bien algunas de estas aplicaciones ya existían antes de la invención de la válvula, su desarrollo se aceleró notablemente gracias a ella [2].

A continuación se enumeran algunos de los avances más notables de la computación basados en el uso de válvulas electrónicas:

- Alan Turing (1912–1954). Contribuyó al desarrollo de COLOSSUS, la primera computadora electrónica, construida por el gobierno británico para descifrar códigos durante la Segunda Guerra Mundial.
- John Mauchly (1907–1980). Participó en la construcción de la ENIAC (Electronic Numerical Integrator And Computer) en 1946, considerada la primera computadora digital, electrónica, de propósito general y programable (ver Figura 12). Contaba con 18 000 válvulas y 1 500 relés, pesaba 30 toneladas y consumía 140 kW. Operaba en sistema decimal. En la Figura 13 se muestra una válvula de vacío.
- EDSAC (Electronic Delay Storage Automatic Calculator). Máquina sucesora de la ENIAC, construida en 1949. A diferencia de esta, operaba en sistema binario y fue una de las primeras computadoras en ejecutar programas almacenados en memoria.
- EDVAC (Electronic Discrete Variable Automatic Computer). Una de las primeras computadoras electrónicas, desarrollada en 1951. A diferencia de la ENIAC, usaba aritmética binaria y fue diseñada siguiendo el concepto de programa almacenado, inspirado en la Máquina de Turing.
- John von Neumann (1903–1957). Desarrolló en 1952 la máquina IAS, en la que se implementó el diseño arquitectónico que hoy se conoce como **máquina de Von Neumann**, compuesto por cinco partes básicas: memoria, unidad aritmético-lógica, unidad de control, entrada y salida. Esta arquitectura continúa siendo la base de casi todas las computadoras digitales actuales.
- Whirlwind I (1951). Diseñada en el MIT, fue una de las primeras computadoras en operar en tiempo real. Utilizaba palabras de 16 bits y representó un avance significativo en aplicaciones de control.

Observación

Las computadoras ENIAC, COLOSSUS, MARK I y MARK II son ejemplos de máquinas con *programa cableado*. Estas no contaban con memoria para almacenar instrucciones; por lo

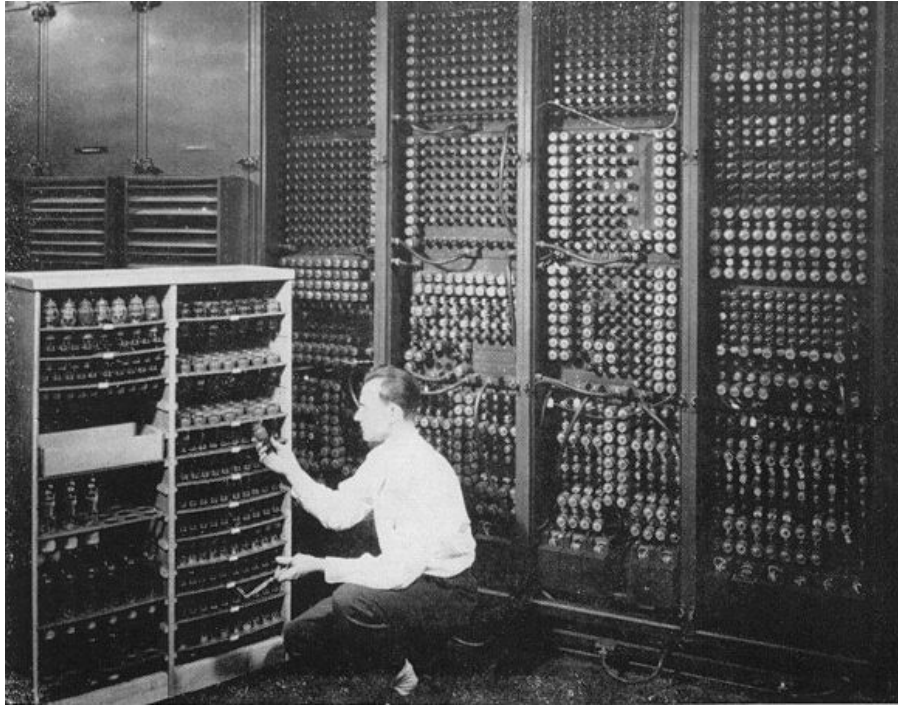


Figura 12: Imagen de la computadora ENIAC (fuente: [3]).



Figura 13: Imagen de una válvula electrónica (fuente: [2]).

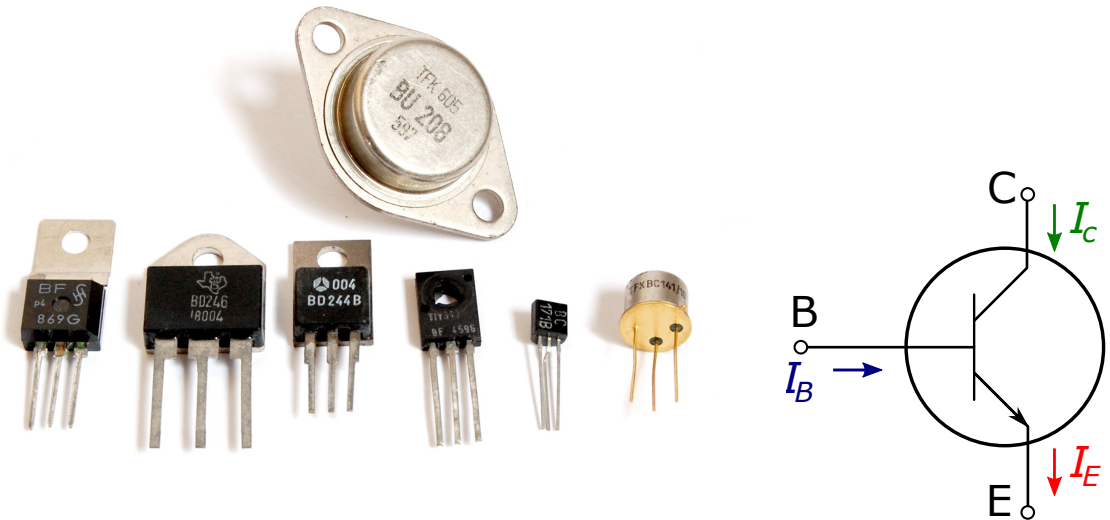
tanto, cada nuevo programa requería una reconfiguración manual del sistema mediante cables y conmutadores. Este proceso era lento y complejo, y limitaba considerablemente la flexibilidad de uso. La posterior adopción del concepto de *programa almacenado*, implementado en máquinas como la EDVAC y la EDSAC, representó un avance fundamental en la evolución de la computación, al permitir modificar y ejecutar programas directamente desde la memoria sin necesidad de realizar cambios físicos en el hardware.

9.3. Computadoras con transistores

En 1948, John Bardeen, Walter Brattain y William Shockley inventaron el transistor mientras trabajaban en los Laboratorios Bell. El transistor es un dispositivo electrónico semiconductor que entrega una señal de salida en respuesta a una señal de entrada. Puede funcionar como amplificador, oscilador, interruptor o rectificador. Actualmente, está presente en prácticamente

todos los aparatos electrónicos de uso diario, tales como radios, televisores, reproductores de audio y video, relojes de cuarzo, computadoras, lámparas fluorescentes, tomógrafos y teléfonos celulares, aunque casi siempre dentro de los llamados circuitos integrados [4].

Los transistores pueden usarse como interruptores, al igual que las válvulas, pero con un tamaño mucho menor, menor costo y menor disipación de calor. En la Figura 14(a) se muestran diferentes tipos de transistores, mientras que en la Figura 14(b) se ilustra el esquema de un transistor tipo NPN, donde se observan sus tres partes fundamentales: la *base*, el *colector* y el *emisor*.



(a) Imagen de algunos tipos de transistores.

(b) Diagrama de Transistor tipo NPN.

Figura 14: Transistores (fuente: [4]).

El impacto del transistor ha sido enorme, ya que, además de dar origen a la industria de los semiconductores, fue el precursor de otros inventos como los circuitos integrados, los dispositivos optoelectrónicos y los microprocesadores. Sin embargo, los cambios más significativos se produjeron en el campo de la computación. Según Albert P. Malvino [5], “*el transistor no se creó para mejorar la computación, sino que fue él quien la creó*”.

Antes de 1950, una computadora ocupaba una sala entera y costaba millones de dólares. Con la incorporación del transistor, fue posible reducir notablemente tanto el tamaño como el costo de estas máquinas. Hoy en día, una computadora cabe en un bolsillo, tiene un costo muy bajo y ofrece enormes prestaciones.

A continuación, se enumeran algunos de los avances más notables en computación que utilizaron esta tecnología:

- TX-0 (1956). Primera computadora transistorizada, desarrollada en el Lincoln Laboratory del MIT.
- IBM 1401 (1959). Computadora desarrollada por IBM, orientada a aplicaciones comerciales y contables (ver Figura 15).
- PDP-1 (1960). Primera microcomputadora; contaba con 4K palabras de 18 bits y un tiempo de ciclo de $5 \mu s$.
- IBM 7094 (1962). Computadora desarrollada por IBM, orientada a la computación científica.

- CDC 6600 (1964). Desarrollada por Control Data Corporation (CDC), considerada la primera supercomputadora científica.

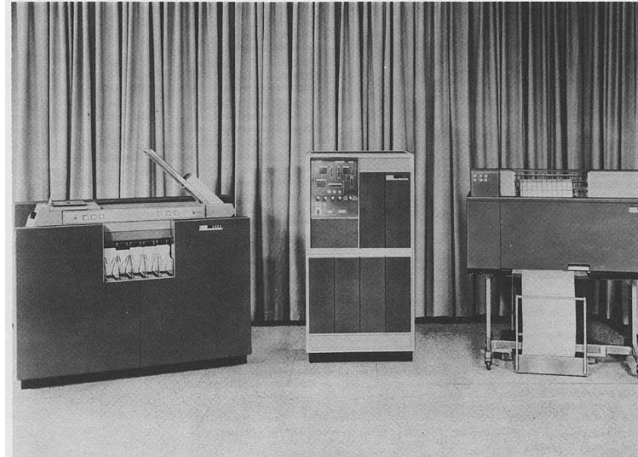


Figura 15: Imagen de la computadora IBM 1401 en el centro; a la izquierda, se observa la lectora/perforadora de tarjetas IBM 1402; a la derecha, la impresora 1403 (fuente: [6]).

9.4. Computadoras con circuitos integrados

En 1958, Robert Noyce inventó el circuito integrado de silicio, lo que permitió colocar decenas de transistores en un solo chip. Un circuito integrado, también conocido como chip o microchip, es una estructura de pequeñas dimensiones hecha de material semiconductor —normalmente silicio— con una superficie de pocos milímetros cuadrados, sobre la cual se fabrican circuitos electrónicos generalmente mediante fotolitografía. Este está protegido dentro de un encapsulado de plástico o cerámica, que cuenta con conductores metálicos adecuados para conectar el circuito integrado a un circuito impreso [7].

La integración de grandes cantidades de transistores diminutos en un espacio reducido representó un avance significativo respecto a la elaboración manual de circuitos con componentes discretos. La producción masiva de circuitos integrados, junto con su fiabilidad y la posibilidad de diseñar circuitos cercanos a un diagrama a bloques, facilitó la rápida adopción de circuitos integrados estandarizados en lugar de los diseños basados en transistores discretos.

A continuación, se enumeran algunos de los avances más notables en computación que utilizaron esta tecnología:

- System/360 (1964). Familia de computadoras desarrollada por IBM, destinada tanto a aplicaciones científicas como comerciales. Una innovación destacada fue la multiprogramación (ver Figura 16).
- PDP-8 (1965). Primera minicomputadora con éxito comercial masivo, con unas 50 000 unidades vendidas.
- PDP-11 (1970). Dominó el mercado de las minicomputadoras durante la década de los setenta.

9.5. Computadoras con integración a muy gran escala

La integración a escala muy grande, o VLSI (siglas en inglés de *very-large-scale integration*), es el proceso de fabricar circuitos integrados que contienen cientos de miles de transistores en un único chip. Esta tecnología comenzó a utilizarse en la década de 1970, como parte de los avances en semiconductores y comunicaciones que se estaban desarrollando [9].

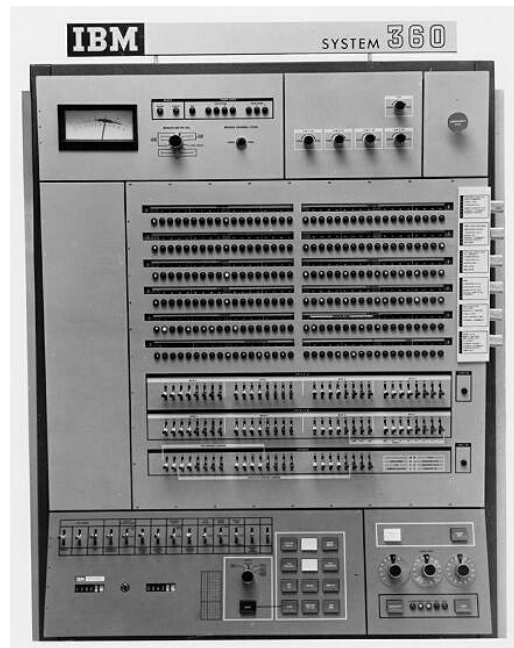


Figura 16: Imagen de la computadora IBM 360 (fuente: [8]).

- Intel 8080 (1974). Primer microprocesador de propósito general de 8 bits en un solo chip. Funcionaba a 2 MHz y es considerado el primer diseño de microprocesador verdaderamente usable. En la Figura 17 se muestra una imagen del microprocesador Intel 8080.



Figura 17: Microprocesador Intel 8080 (fuente: [10]).

- Apple II (1977). Primera serie de microcomputadoras de producción masiva desarrollada por Apple Computer (ver Figura 18). Contaba con una arquitectura de 8 bits.
- 8086 (1978). Uno de los primeros microprocesadores de 16 bits en un solo chip, con 29 000 transistores. Su variante, el 8088, con un bus externo de 8 bits, fue utilizado en la computadora personal original de IBM.
- 8087 (1980). Intel introdujo el coprocesador de punto flotante 8087 (45 000 transistores), diseñado para trabajar junto al procesador 8086 o 8088 y ejecutar instrucciones de punto flotante. El 8087 estableció el modelo de punto flotante para la línea x86, conocido comúnmente como “x87”.
- IBM Personal Computer (1981). Se convirtió en la computadora más vendida de la historia. Venía equipada con el sistema operativo MS-DOS, provisto por Microsoft Corporation.
- 80286 (1982). Añadió más modos de direccionamiento de memoria y formó la base para la IBM PC-AT, la plataforma original para MS Windows. Contaba con registros de 16 bits y 134 000 transistores.
- RISC-I (1982). Desarrollada en el proyecto RISC de la Universidad de Berkeley, bajo la dirección de David A. Patterson. RISC (Reduced Instruction Set Computer) propone reemplazar arquitecturas complejas (CISC) por otras más sencillas y rápidas.



Figura 18: Computadora Apple II (fuente: [11]).

- R2000 (1985). Primer diseño MIPS creado por John L. Hennessy, que mejoró drásticamente el rendimiento mediante el uso de segmentación.
- i386 (1985). Expandió la arquitectura x86 a 32 bits. Contaba con 275 000 transistores.
- i486 (1989). Mejoró el desempeño de la línea x86 e integró la unidad de punto flotante en el chip, aunque sin cambios significativos en el conjunto de instrucciones. Tenía 1.2 millones de transistores.
- Pentium (1993). Mejoró el desempeño y añadió extensiones menores al conjunto de instrucciones. Contaba con 3.1 millones de transistores.
- Pentium Pro (1995). Introdujo una microarquitectura radicalmente nueva, conocida internamente como P6. Añadió instrucciones de movimiento condicional al conjunto. Poseía 5.5 millones de transistores.
- Pentium II (1997). Continuó con la microarquitectura P6. Contaba con 7 millones de transistores.
- Pentium III (1999). Introdujo SSE, un conjunto de instrucciones para manipular vectores de datos enteros o de punto flotante. Tenía 8.2 millones de transistores, con versiones posteriores que llegaron a 24 millones.
- Pentium 4 (2000). Extendió SSE a SSE2, incorporando nuevos tipos de datos, incluyendo punto flotante de doble precisión, junto con numerosas instrucciones adicionales. Contaba con 42 millones de transistores.
- Pentium 4E (2004). Añadió *hyperthreading*, que permite ejecutar dos programas simultáneamente en un solo procesador, así como EM64T, la implementación de Intel de una extensión de 64 bits para IA-32 desarrollada por AMD, conocida como x86-64. Tenía 125 millones de transistores.
- Core 2 (2006). Retornó a una microarquitectura similar a P6. Fue el primer microprocesador multinúcleo de Intel, integrando múltiples núcleos en un solo chip. No es compatible con *hyperthreading*. Poseía 291 millones de transistores.

- Core i7 (2008). Incorporó tanto *hyperthreading* como *multi-core*, con la versión inicial que permitía ejecutar dos hilos por núcleo y hasta cuatro núcleos por chip. Contaba con 781 millones de transistores.

A. Apéndices

A.1. Primeros lenguajes de programación

A continuación, se presentan los primeros lenguajes de programación junto con una descripción de sus características fundamentales:

Lenguaje	Año	Principales características
Assembly	1940s	Lenguaje de bajo nivel, cercano al código máquina; específico para cada arquitectura; manipulación directa de registros y memoria.
FORTRAN	1957	Primer lenguaje de alto nivel ampliamente usado; orientado a cálculos científicos y matemáticos; facilitaba la programación estructurada y la portabilidad.
LISP	1958	Primer lenguaje para inteligencia artificial; basado en listas; soporta recursión y manejo simbólico.
COBOL	1959	Orientado a aplicaciones de negocio; diseñado para ser legible y cercano al lenguaje inglés; usado en sistemas administrativos y financieros.
ALGOL	1958-1960	Introdujo conceptos de lenguajes estructurados y de bloques; influyó en muchos lenguajes posteriores como Pascal, C y Java.
BASIC	1964	Diseñado para principiantes; fácil de aprender y usar; orientado a educación y programación interactiva.
PL/I	1964	Intentó unificar características de FORTRAN y COBOL; diseñado para aplicaciones científicas y comerciales; lenguaje complejo.
Simula	1967	Primer lenguaje orientado a objetos; basado en ALGOL; usado para simulaciones y modelado.
Pascal	1970	Diseñado con fines educativos; promueve la programación estructurada y el uso riguroso de tipos de datos. Fue ampliamente utilizado en enseñanza y desarrollo de software en los años 80.
C	1972	Diseñado originalmente para escribir sistemas operativos como UNIX, es un lenguaje ampliamente utilizado que aún hoy se emplea de forma masiva. Ha influido en el diseño de muchos lenguajes modernos, como C++, Java y Python.

Referencias

- [1] Historia de la Informática. Harvard MARK I (1944), 2015. [Internet; descargado 2-agosto-2023].
- [2] Colaboradores de Wikipedia. Válvula termoiónica — Wikipedia, La enciclopedia libre. https://es.wikipedia.org/w/index.php?title=V%C3%A1lvula_termoi%C3%B3nica&oldid=121814534, 2019. [Internet; descargado 27-enero-2020].

- [3] Historia de la Informática. Proyecto ENIAC, 2011. [Internet; descargado 31-enero-2022].
- [4] Colaboradores de Wikipedia. Transistor — Wikipedia, La enciclopedia libre. <https://es.wikipedia.org/w/index.php?title=Transistor&oldid=122923620>, 2020. [Internet; descargado 27-enero-2020].
- [5] Albert Paul Malvino and David J. Batesr. *Principios de Electrónica*. Mcgraw-Hill, 2007.
- [6] Colaboradores de Wikipedia. IBM 1401 — Wikipedia, La enciclopedia libre, 2021. [Internet; descargado 31-enero-2022].
- [7] Colaboradores de Wikipedia. Circuito integrado — Wikipedia, La enciclopedia libre. https://es.wikipedia.org/w/index.php?title=Circuito_integrado&oldid=122233649, 2019. [Internet; descargado 27-enero-2020].
- [8] Colaboradores de Wikipedia. IBM System/360, 2015. [Internet; descargado 2-agosto-2021].
- [9] Colaboradores de Wikipedia. Integración a muy gran escala — Wikipedia, La enciclopedia libre. https://es.wikipedia.org/w/index.php?title=Integraci%C3%B3n_a_muy_gran_escala&oldid=117810308, 2019. [Internet; descargado 27-enero-2020].
- [10] Elprocus. Introduction to 8080 Microprocessor and its Architecture, 2015. [Internet; descargado 2-agosto-2023].
- [11] Colaboradores de Wikipedia. Apple II, 2023. [Internet; descargado 2-agosto-2023].
- [12] Ed Jorgenson. *X86-64 Assembly Language Programming with Ubuntu*. 2019.
- [13] John L Hennessy and David A Patterson. *Computer architecture: A quantitative approach*. Elsevier, 2011.
- [14] Randal E Bryant and David Richard O'Hallaron. *Computer systems: a programmer's perspective*, volume 281. Prentice Hall Upper Saddle River, second edition, 2003.
- [15] Richard Blum. *Professional assembly language*. John Wiley & Sons, 2007.
- [16] Andrew S Tanenbaum. *Organización de computadoras: un enfoque estructurado*. Pearson educación, 2000.
- [17] Randall Hyde. *The art of assembly language*. No Starch Press, 2003.
- [18] Igor Zhirkov. *Low-Level Programming: C, Assembly, and Program Execution on Intel 64 Architecture*. Apress, 2017.
- [19] Sarah L Harris and David Harris. *Digital design and computer architecture*. Morgan Kaufmann, 2015.
- [20] William Stallings. *Computer organization and architecture: designing for performance*. Pearson Education India, 11th edition, 2022.